

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для автоматизации учета книг и читателей библиотеки»

Выполнил: Лендер Марк Сергеевич
Группа: 241-3211

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Лендеру Марку Сергеевичу,

обучающемуся группы 241-3211,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для автоматизации учета книг и читателей библиотеки»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих систем и программных продуктов	10	22.03.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	26.03.2026	
Задача 1.3. Формулировка цели и задач работы	8	30.03.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории	5	02.04.2026	
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования, user story)	7	06.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	10.04.2026	
Задача 2.4. Разработка макетов страниц (Wireframe)	5	14.04.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры приложения и вёрстка шаблонов страниц	12	21.04.2026	
Задача 3.2. Реализация модуля авторизации и разграничения прав доступа (библиотекарь/читатель)	8	27.04.2026	
Задача 3.3. Разработка CRUD-интерфейса для управления книгами, читателями и журналом выдач	9	03.05.2026	
Задача 3.4. Реализация системы поиска и фильтрации книг, формирование статистики	11	09.05.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	22.03.2026	
Задача 4.2. Деплой приложения на хостинг	4	26.05.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	26.05.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____ 2026 г. _____ А. С. Кружалов
(подпись)

Дата выдачи задания _____ «03» апреля 2026 г.

Дата сдачи выполненной работы _____ «____» _____ 2026 г.

Задание принял к исполнению

«03» апреля 2026 г. _____ М. С. Лендер
(подпись)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Обзор существующих систем и программных продуктов	6
1.2 Анализ программных инструментов разработки веб-приложений	8
1.3 Формулировка цели и задач работы	11
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	13
2.1 Анализ целевой аудитории	13
2.2 Описание функциональности приложения	14
2.3 Проектирование модели данных	17
2.4 Разработка макетов страниц	22
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	25
ЗАКЛЮЧЕНИЕ	26

ВВЕДЕНИЕ

Актуальность темы. Библиотеки хранят и обрабатывают значительный объем информации о книжном фонде, читателях, выдаче и возврате книг. При ручном ведении учета или использовании разрозненных таблиц возникают проблемы с поиском данных, контролем доступности книг, учетом задолженностей и формированием статистики. Разработка веб-приложения для автоматизации учета книг и читателей позволяет упростить работу библиотекаря и сделать информацию о фонде более доступной для читателей.

Объект исследования — процессы учета книг, читателей и выдачи книг в библиотеке.

Предмет исследования — веб-приложение для автоматизации учета книг и читателей библиотеки.

Цель работы — разработать веб-приложение для автоматизации учета книг и читателей библиотеки.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Выполнить анализ предметной области и существующих программных решений.
2. Выбрать программные инструменты разработки веб-приложения.
3. Спроектировать структуру приложения и модель данных.
4. Реализовать аутентификацию, авторизацию и разграничение прав доступа.
5. Реализовать CRUD-интерфейс для книг, читателей и журнала выдач.
6. Реализовать поиск, фильтрацию, статистику, загрузку и выгрузку файлов.
7. Подготовить отчет о проделанной работе.

Практическая значимость работы заключается в создании веб-приложения, демонстрирующего основные принципы разработки информационной системы: работу с пользователями, базой данных, интерфейсом, файлами и правами доступа.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих систем и программных продуктов

Библиотека как объект автоматизации включает несколько взаимосвязанных процессов: учет книжного фонда, регистрацию читателей, выдачу и возврат книг, контроль задолженностей, поиск изданий, а также формирование отчетов по движению фонда. При ручном ведении учета, например в бумажных журналах или отдельных электронных таблицах, возникают типовые проблемы: дублирование записей, сложность поиска нужного издания, отсутствие оперативной статистики, риск потери истории выдач, а также невозможность разграничить доступ между сотрудником библиотеки и обычным читателем.

В рамках данной работы под небольшой библиотекой понимается библиотека с ограниченным объемом книжного фонда, небольшим количеством читателей и одним или несколькими сотрудниками, выполняющими функции библиотекаря. Для такой библиотеки основными задачами являются учет книг, регистрация читателей, оформление выдачи и возврата книг, поиск изданий и получение простой статистики. При этом сложные механизмы профессиональной каталогизации, комплектования фонда, интеграции с внешними библиотечными системами и распределения работы между несколькими специализированными отделами могут быть избыточными.

Для решения задач автоматизации библиотечного учета существуют автоматизированные библиотечно-информационные системы. Они позволяют вести электронные каталоги, учитывать читателей, фиксировать выдачу и возврат книг, а также формировать отчеты. Рассмотрим несколько характерных решений: KoHa, ИРБИС64 и 1С:Библиотека.

KoHa является свободной и открытой библиотечной системой, предназначенной для автоматизации основных библиотечных процессов. Система включает средства ведения каталога, обслуживания читателей, учета выдач и формирования отчетов [1]. Данное решение является функционально развитым и может использоваться различными типами библиотек. При этом внедрение KoHa требует настройки серверной инфраструктуры, а также последующего администрирования и сопровождения системы.

САБ ИРБИС64 представляет собой российское решение для автоматизации библиотечно-информационных технологических процессов. В состав системы входят серверная часть и автоматизированные рабочие места, вклю-

чая каталогизацию, комплектование, книговыдачу и другие модули [2]. Данная система ориентирована на профессиональную библиотечную деятельность и поддерживает широкий набор специализированных процессов. Для небольшой библиотеки такая функциональность может быть избыточной, так как требует освоения отдельных модулей и настройки системы под конкретные процессы.

Решение 1С:Библиотека предназначено для автоматизации деятельности библиотек разных типов и назначения [3]. Его особенностью является работа в экосистеме 1С и возможность использования типовых учетных механизмов. Такое решение удобно для организаций, уже использующих платформу 1С, однако требует наличия соответствующей инфраструктуры и сопровождения. Кроме того, зависимость от платформы 1С снижает гибкость при разработке индивидуального веб-интерфейса.

Сравнение рассмотренных решений по основным критериям приведено в таблице 1.1.

Таблица 1.1 – Сравнение существующих систем автоматизации библиотек

Система	Назначение	Сложность установки	Сложность сопровождения	Применимость для небольшой библиотеки
Koha	Открытая АБИС для комплексной автоматизации библиотек	Средняя или высокая: требуется серверное окружение и настройка	Средняя или высокая: требуется администрирование, обновление и настройка	Подходит, но может быть избыточна для базового учета книг и выдач
ИРБИС64	Российская профессиональная АБИС для библиотечно-информационных процессов	Средняя или высокая: используется серверная часть и отдельные рабочие места	Высокая: требуется настройка модулей и сопровождение специализированных процессов	Подходит для профессиональных библиотек, но избыточна для базовых процессов учета
1С:Библиотека	Учетное решение для автоматизации библиотек на платформе 1С	Средняя: требуется платформа 1С и настройка конфигурации	Средняя или высокая: требуется сопровождение платформы и обновление конфигурации	Подходит при наличии инфраструктуры 1С, но менее удобна как самостоятельное веб-решение

Проведенный анализ показывает, что существующие системы в основном

ориентированы на комплексную автоматизацию библиотек. Они содержат развитые средства каталогизации, поддержки библиотечных стандартов, работы с электронными ресурсами и формирования отчетности. Такие возможности являются полезными для крупных библиотек, однако для небольшой библиотеки они могут быть избыточными и приводить к усложнению внедрения, настройки и дальнейшего сопровождения системы.

В связи с этим целесообразна разработка более простого веб-приложения, ориентированного на базовые процессы библиотечного учета: ведение списка книг, учет читателей, регистрацию выдачи и возврата книг, поиск и фильтрацию записей, а также получение простой статистики. Отличие разрабатываемого решения заключается в упрощенной структуре, ориентации на наиболее востребованные функции и отсутствии избыточных модулей, необходимых только для крупных библиотек или профессиональных библиотечно-информационных систем.

1.2 Анализ программных инструментов разработки веб-приложений

Для разработки веб-приложения выбран стек Python, Django, SQLite, HTML, CSS и JavaScript. Такой набор технологий позволяет реализовать серверную часть, пользовательский интерфейс, хранение данных, работу с файлами и разграничение доступа в рамках единой архитектуры.

Python выбран в качестве основного языка программирования серверной части. К его преимуществам относятся читаемый синтаксис, большое количество библиотек, активная экосистема и широкое применение в веб-разработке. В качестве альтернативы мог бы использоваться JavaScript на серверной стороне, однако в таком случае потребовалась бы отдельная настройка серверной платформы и дополнительных библиотек для реализации авторизации, работы с базой данных и шаблонами. Язык C++ также может применяться для серверной разработки, но для создания типового веб-приложения с формами, базой данных и пользовательскими ролями он требует больше низкоуровневой настройки и менее удобен для быстрой реализации прикладной бизнес-логики. Поэтому для данного приложения выбран Python как более удобный язык для разработки серверной логики учета книг, читателей, выдач и возвратов.

В качестве веб-фреймворка выбран Django. Django является высоко-

уровневым Python-фреймворком, ориентированным на быструю разработку веб-приложений и чистую структуру проекта [4]. Одним из важных преимуществ Django является наличие встроенных механизмов, которые часто требуются в информационных системах: работа с пользователями, сессиями, правами доступа, формами, базой данных и административным интерфейсом.

Альтернативой Django является Flask. Flask является микрофреймворком для Python, который предоставляет базовые средства маршрутизации и обработки запросов [5]. Flask удобен для небольших приложений и дает разработчику больше свободы в выборе компонентов. Однако для реализации полноценной системы учета с авторизацией, ролями, CRUD-интерфейсом и формами потребовалось бы дополнительно подключать и настраивать отдельные расширения, например для авторизации, работы с базой данных и миграциями. В Django значительная часть этих механизмов уже предусмотрена, поэтому он лучше подходит для разрабатываемого приложения.

Для разрабатываемого приложения особенно важна система аутентификации и авторизации. Django содержит встроенную систему пользователей, групп, прав и сессий [6]. Это позволяет реализовать роли, например «библиотекарь» и «читатель», а также ограничить доступ к отдельным действиям: созданию и редактированию книг, оформлению выдач, просмотру статистики и управлению читателями.

Для хранения данных выбрана SQLite. SQLite является самодостаточным SQL-движком, не требующим отдельного серверного процесса и сложной настройки [7]. База данных хранится в одном файле, что упрощает развертывание и сопровождение приложения. Такой вариант подходит для небольшой библиотеки, где количество пользователей ограничено, а операции записи выполняются сравнительно редко.

Альтернативой SQLite является PostgreSQL. PostgreSQL представляет собой полноценную объектно-реляционную систему управления базами данных с открытым исходным кодом [8]. PostgreSQL лучше подходит для систем с большим количеством пользователей, высокой нагрузкой, активной одновременной записью данных и более сложным администрированием. Однако использование PostgreSQL требует установки и сопровождения отдельного сервера баз данных, создания пользователей, настройки прав доступа и резервного копирования. Для небольшой библиотеки, которой требуется простое

развертывание и базовый учет, SQLite является более простым вариантом. При этом SQLite поддерживает основные возможности реляционной базы данных, необходимые для приложения: создание таблиц, хранение связанных данных, выполнение запросов, фильтрацию и агрегирование данных.

Работа с базой данных будет выполняться через ORM Django. Это позволит описывать модели приложения на языке Python, а затем автоматически создавать таблицы базы данных с помощью миграций. При необходимости дальнейшего развития приложения использование ORM упростит перенос проекта с SQLite на другую реляционную СУБД, например PostgreSQL.

Для реализации пользовательского интерфейса используются HTML, CSS и JavaScript. HTML задает структуру страниц, CSS отвечает за внешний вид, а JavaScript может использоваться для небольших интерактивных элементов: подтверждения действий, динамического отображения данных, улучшения форм и фильтров.

Для формирования HTML-страниц выбран подход с использованием серверных шаблонов Django. Альтернативой мог бы быть отдельный frontend-фреймворк, например React или Vue, при котором серверная часть предоставляет API, а пользовательский интерфейс разрабатывается как отдельное клиентское приложение. Такой подход целесообразен для более сложных интерактивных систем, но для приложения учета книг, читателей и выдач он усложнил бы архитектуру. Использование шаблонов Django позволяет реализовать интерфейс проще: сервер сразу формирует готовые HTML-страницы, а JavaScript применяется только там, где требуется небольшая интерактивность.

Также в приложении предусматривается работа с файлами. Django содержит штатные средства обработки файлов, загружаемых через формы, включая работу с объектом `request.FILES` [9]. В рамках приложения загрузка файлов может использоваться для добавления обложек книг, а выгрузка данных — для формирования отчетов по книжному фонду и журналу выдач.

Сравнение выбранных инструментов и рассмотренных альтернатив приведено в таблице 1.2.

Таблица 1.2 – Сравнение инструментов разработки

Компонент	Выбранное решение	Рассмотренные альтернативы	Причина выбора
Язык серверной части	Python	JavaScript на серверной стороне, C++	Python удобен для реализации прикладной бизнес-логики и имеет развитую экосистему для веб-разработки
Веб-фреймворк	Django	Flask	Django содержит встроенные механизмы пользователей, прав доступа, форм, ORM и административного интерфейса
База данных	SQLite	PostgreSQL	SQLite проще в развертывании и сопровождении, так как не требует отдельного сервера БД и подходит для небольшой библиотеки
Формирование интерфейса	HTML, CSS, шаблоны Django, JavaScript	Отдельный frontend-фреймворк, например React или Vue	Серверные шаблоны проще для приложения с типовыми страницами, формами и таблицами
Работа с файлами	Средства Django для загрузки файлов	Ручная обработка файлов без встроенных механизмов фреймворка	Django предоставляет готовые средства обработки загруженных файлов через формы

Таким образом, выбранный стек технологий позволяет разработать веб-приложение с понятной структурой, ролевой моделью доступа, интерфейсом управления данными, поиском, фильтрацией, статистикой и обработкой файлов. При этом архитектура остается достаточно простой для развертывания, сопровождения и возможного дальнейшего расширения.

1.3 Формулировка цели и задач работы

По результатам анализа предметной области установлено, что для небольших библиотек может быть востребовано веб-приложение, ориентированное не на комплексную профессиональную автоматизацию всех библиотечных процессов, а на решение базовых задач учета книг, читателей и выдач. Такое приложение должно быть проще во внедрении и сопровождении, чем полно-

функциональные библиотечно-информационные системы, но при этом должно покрывать основные операции ежедневной работы библиотеки.

Целью разработки является создание веб-приложения для автоматизации учета книг и читателей библиотеки, обеспечивающего ведение книжного фонда, учет читателей, регистрацию выдачи и возврата книг, поиск информации и формирование статистики.

Для достижения указанной цели необходимо решить следующие задачи:

1. определить основные процессы предметной области и состав пользователей приложения;
2. спроектировать модель данных для хранения информации о книгах, читателях, авторах, категориях и выдачах;
3. реализовать механизм регистрации, входа в систему и разграничения доступа пользователей по ролям;
4. разработать интерфейсы для добавления, просмотра, редактирования и удаления записей о книгах, читателях и выдачах;
5. реализовать поиск и фильтрацию книг по основным параметрам;
6. реализовать обработку данных и формирование статистики по книжному фонду и журналу выдач;
7. реализовать загрузку и выгрузку файлов, связанных с данными приложения;
8. выполнить проверку работоспособности основных функций веб-приложения.

В качестве основных сущностей приложения предполагается использовать следующие: Book — книга, Reader — читатель, Loan — запись о выдаче книги. Дополнительно могут быть выделены сущности Author, Category и UploadedFile. Такая структура данных позволяет описать основные объекты предметной области и связать учет книг с действиями читателей.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Разрабатываемое веб-приложение предназначено для небольшой библиотеки, в которой основными процессами являются учет книг, регистрация читателей, оформление выдачи и возврата книг, поиск изданий и получение простой статистики. Приложение предполагает работу через браузер, поэтому пользователю не требуется устанавливать отдельную клиентскую программу на рабочий компьютер.

Основными пользователями системы являются:

- **Библиотекарь** — сотрудник библиотеки, ответственный за ведение книжного фонда, регистрацию читателей, оформление выдачи и возврата книг, а также просмотр статистики.
- **Читатель** — пользователь библиотеки, которому необходимо просматривать каталог книг, искать нужные издания и отслеживать информацию о собственных выдачах.
- **Администратор** — технический пользователь, отвечающий за управление учетными записями, назначение ролей и контроль работы приложения.

В небольшой библиотеке роли администратора и библиотекаря могут выполняться одним сотрудником. При этом в проектируемой системе они разделяются логически: администратор отвечает за учетные записи, роли и права доступа, а библиотекарь работает с предметными данными — книгами, читателями и выдачами. Такое разделение позволяет сохранить простую структуру приложения и одновременно явно определить границы доступа пользователей.

Для библиотекаря основными потребностями являются быстрый доступ к операциям добавления и редактирования книг, оформление выдач, контроль возвратов и получение сводной информации о состоянии фонда. Для читателя важны простота поиска, доступность каталога и возможность видеть сведения о выданных ему книгах. Администратор обеспечивает техническую настройку системы и разграничение прав доступа.

Таким образом, интерфейс приложения должен быть разделен по ролям. Библиотекарь получает доступ к разделам управления книгами, читателями и журналом выдач. Читатель получает доступ к каталогу и личной информации о выдачах. Администратор получает доступ к управлению пользователями и ролями.

2.2 Описание функциональности приложения

Функциональность веб-приложения определяется основными процессами библиотеки. Система должна обеспечивать учет книг, учет читателей, оформление выдачи и возврата книг, поиск и фильтрацию записей, формирование статистики, а также загрузку и выгрузку файлов.

Основной пользовательский сценарий для библиотекаря можно сформулировать в виде *User Story*: «Я, как библиотекарь, хочу вести электронный каталог книг и журнал выдач, чтобы быстро находить информацию о книгах, читателях и текущем состоянии фонда».

Для читателя основной пользовательский сценарий формулируется следующим образом: «Я, как читатель, хочу просматривать каталог и искать книги по названию, автору или категории, чтобы заранее узнать, есть ли нужное издание в библиотеке».

Основные функции приложения:

- регистрация и вход пользователей в систему;
- разграничение прав доступа для ролей «библиотекарь», «читатель» и «администратор»;
- просмотр каталога книг;
- поиск и фильтрация книг по названию, автору, категории и доступности;
- добавление, просмотр, редактирование и удаление записей о книгах;
- добавление, просмотр, редактирование и удаление записей о читателях;
- оформление выдачи книги читателю;
- оформление возврата книги;
- просмотр журнала выдач;

- формирование статистики по книгам, читателям и выдачам;
- загрузка файлов, например обложек книг;
- выгрузка данных для формирования отчетов.

При проектировании функциональности необходимо учитывать, что часть действий доступна только после входа пользователя в систему. В частности, просмотр личных выдач доступен только авторизованному читателю, так как система должна определить, к какой учетной записи относится пользователь. Аналогично, операции управления книгами, читателями, выдачами, файлами и статистикой доступны только пользователю с ролью библиотекаря.

На рисунке 2.1 представлена диаграмма вариантов использования разрабатываемого веб-приложения.

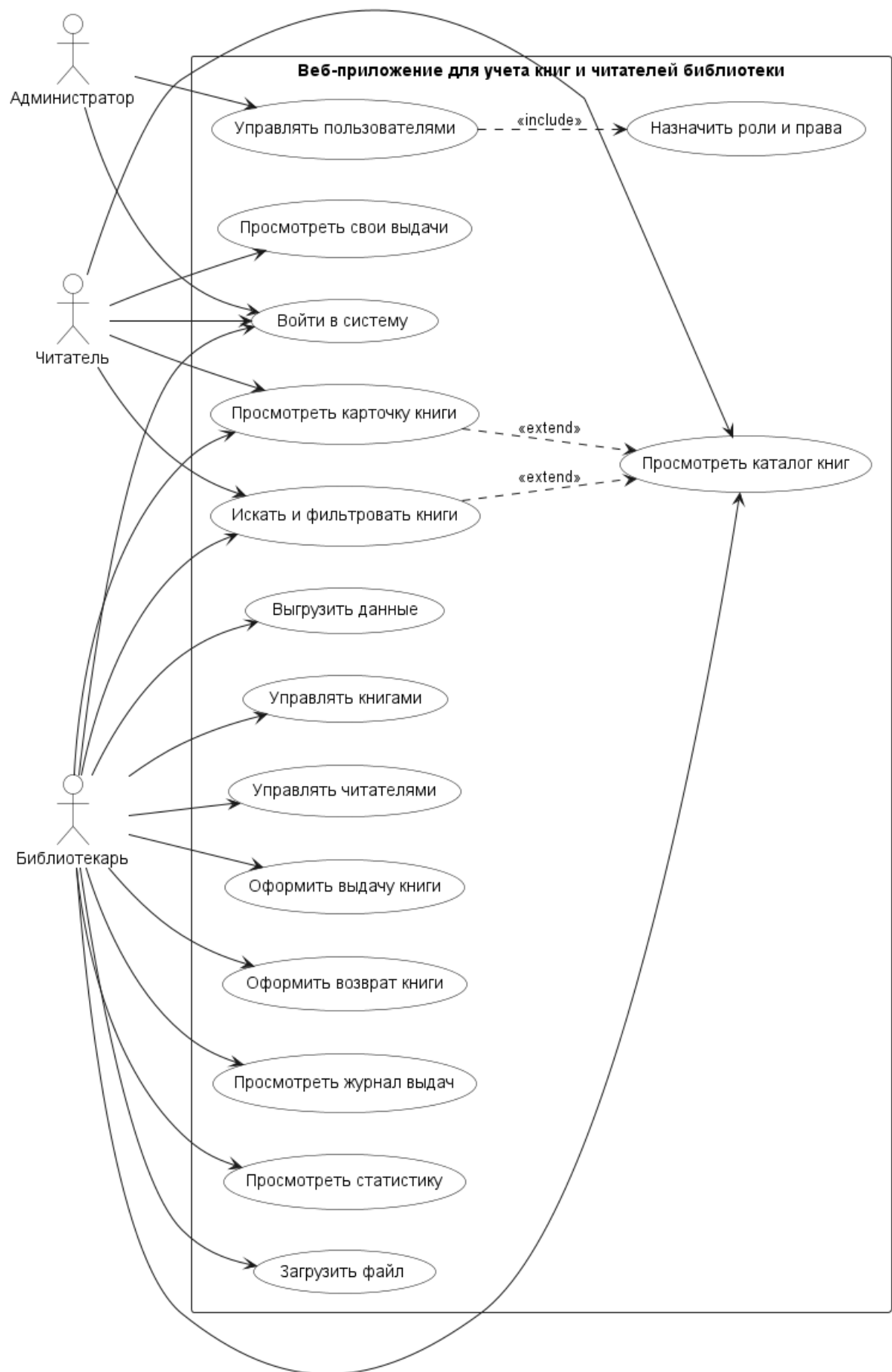


Рисунок 2.1 – Диаграмма вариантов использования веб-приложения

Вариант использования «Загрузить файл» в рамках разрабатываемого приложения предполагает загрузку файлов, связанных с данными системы. Основным пример такой операции — загрузка обложки книги при добавлении или редактировании записи о книге. При дальнейшем развитии приложения данный механизм может использоваться и для загрузки файлов импорта данных.

Вариант использования «Выгрузить данные» означает экспорт сведений из системы во внешний файл. Такая функция может применяться для выгрузки списка книг, журнала выдач или простого отчета по состоянию библиотечного фонда.

Следует также различать управление читателями и управление пользователями. Вариант использования «Управлять читателями» относится к работе библиотекаря с карточками читателей: добавлению читателя, редактированию ФИО, контактных данных и просмотру информации о выданных книгах. Вариант использования «Управлять пользователями» относится к действиям администратора с учетными записями, ролями и правами доступа.

2.3 Проектирование модели данных

Для хранения информации используется реляционная модель данных. Основными сущностями приложения являются книга, читатель и запись о выдаче. Дополнительно выделяются сущности автора, категории и загружаемого файла. Учетные записи пользователей и роли реализуются с использованием встроенных механизмов Django.

Основные сущности модели данных:

- User — учетная запись пользователя системы;
- Reader — читатель библиотеки;
- Author — автор книги;
- Category — категория книги;
- Book — книга библиотечного фонда;
- BookCategory — связь книги и категории;
- Loan — запись о выдаче книги;

- UploadedFile — файл, загруженный пользователем.

Связи между сущностями определяются логикой предметной области. Один автор может быть связан с несколькими книгами. Одна книга может относиться к нескольким категориям, а одна категория может использоваться для классификации нескольких книг. Один читатель может иметь несколько записей о выдаче. Одна книга может многократно фигурировать в журнале выдач, так как она может выдаваться разным читателям в разное время.

На рисунке 2.2 представлена ER-диаграмма модели данных.

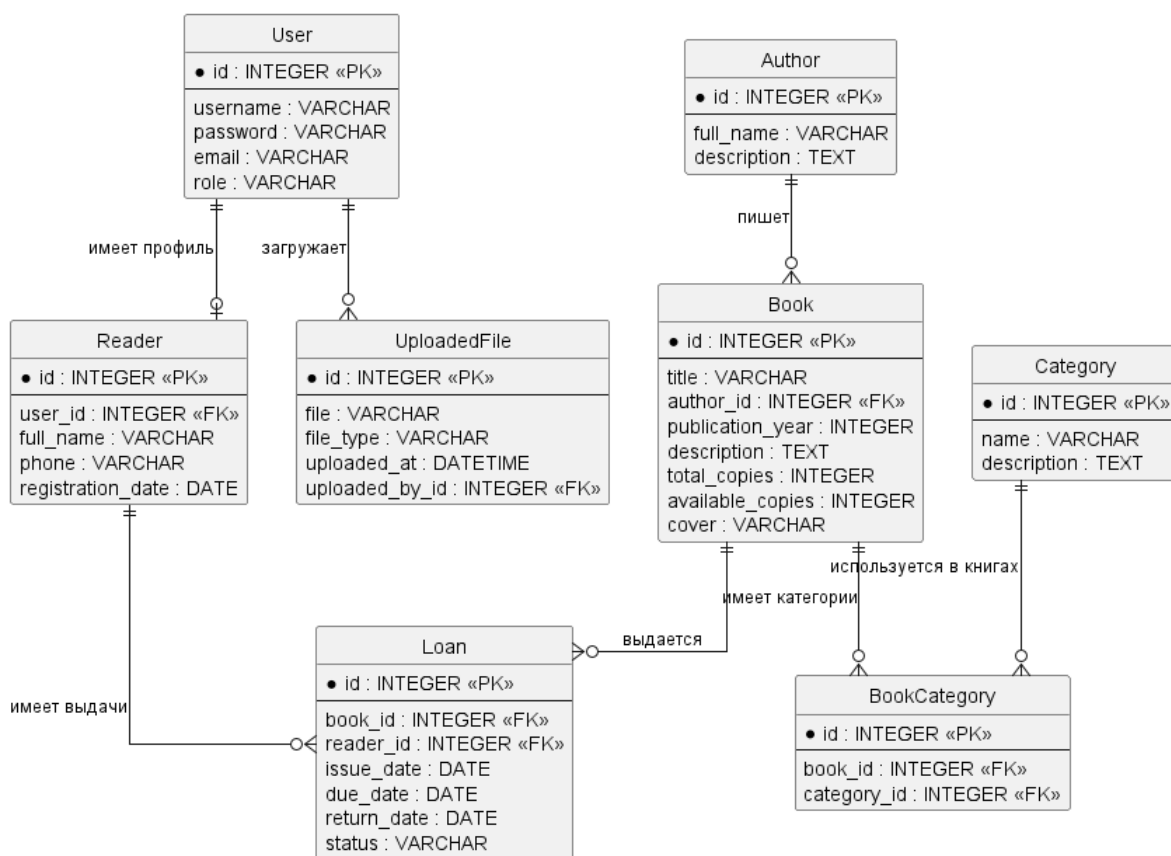


Рисунок 2.2 – ER-диаграмма модели данных

На ER-диаграмме первичные ключи сущностей обозначены как PK, а внешние ключи — как FK. Связи между сущностями показывают логические отношения между объектами предметной области. Конкретные соответствия первичных и внешних ключей дополнительно раскрываются в логической и физической схемах базы данных, представленных в табл. 2.1 и табл. 2.2.

Сущность User используется для хранения учетных записей пользователей: логина, пароля, электронной почты и роли. Сущность Reader описывает карточку читателя библиотеки и связана с учетной записью пользователя через внешний ключ user_id. Такое разделение позволяет отдельно хранить

данные, необходимые для входа в систему, и предметные данные читателя: ФИО, телефон и дату регистрации.

Отдельные сущности для библиотекаря и администратора в модели данных не выделяются, так как данные пользователи отличаются прежде всего набором прав доступа. Их учетные записи хранятся в сущности User, а доступ к функциям системы определяется ролью пользователя. При необходимости дальнейшего развития системы для сотрудников библиотеки можно добавить отдельную сущность профиля, но для базового учета книг, читателей и выдач это является избыточным.

Сущность Book используется для хранения сведений о конкретном издании книги: названии, авторе, годе издания, описании, количестве экземпляров и файле обложки. В каталоге несколько изданий одного произведения могут отображаться как обобщенная запись с возможностью раскрытия списка изданий. При этом на уровне базы данных каждое издание хранится как отдельная запись сущности Book.

Связь между книгами и категориями является связью многие ко многим: одна книга может относиться к нескольким категориям, а одна категория может использоваться для нескольких книг. Для реализации такой связи выделена промежуточная сущность BookCategory, которая связывает записи Book и Category через внешние ключи book_id и category_id.

Сущность Loan используется для хранения записей о выдаче книг. Она связана с сущностью Reader через внешний ключ reader_id, а с сущностью Book — через внешний ключ book_id. Благодаря этому один читатель может иметь несколько записей о выдаче, а одна книга может многократно встречаться в журнале выдач в разные периоды времени.

Сущность UploadedFile используется для хранения сведений о загруженных файлах. В рамках приложения такие файлы могут применяться, например, для загрузки обложек книг или вспомогательных файлов, связанных с импортом данных. Выгрузка данных при этом может выполняться как формирование отдельного файла отчета на основе сведений о книгах, читателях и выдачах.

Логическая схема базы данных представлена в табл. 2.1.

Таблица 2.1 – Логическая схема базы данных

Сущность	Атрибуты	Связи
User	id, username, password, email, role	Один к одному с Reader; один ко многим с UploadedFile
Reader	id, user_id, full_name, phone, registration_date	Один к одному с User; один ко многим с Loan
Author	id, full_name, description	Один ко многим с Book
Category	id, name, description	Один ко многим с BookCategory
Book	id, title, author_id, publication_year, description, total_copies, available_copies, cover	Многие к одному с Author; один ко многим с Loan; один ко многим с BookCategory
BookCategory	id, book_id, category_id	Многие к одному с Book; многие к одному с Category
Loan	id, book_id, reader_id, issue_date, due_date, return_date, status	Многие к одному с Book; многие к одному с Reader
UploadedFile	id, file, file_type, uploaded_at, uploaded_by_id	Многие к одному с User

Физическая схема базы данных будет реализована с использованием моделей Django. Для основных полей будут применяться типы данных, соответствующие возможностям ORM Django и SQLite. Предварительная структура ключевых таблиц представлена в табл. 2.2.

Таблица 2.2 – Физическая схема основных таблиц

Поле	Тип данных	Описание
Таблица readers		
id	INTEGER	Первичный ключ
user_id	INTEGER	Внешний ключ на пользователя
full_name	VARCHAR(255)	ФИО читателя
phone	VARCHAR(20)	Телефон читателя
registration_date	DATE	Дата регистрации читателя
Таблица authors		
id	INTEGER	Первичный ключ
full_name	VARCHAR(255)	ФИО автора
description	TEXT	Краткая информация об авторе

Продолжение таблицы 2.2

Поле	Тип данных	Описание
Таблица categories		
id	INTEGER	Первичный ключ
name	VARCHAR(100)	Название категории
description	TEXT	Описание категории
Таблица books		
id	INTEGER	Первичный ключ
title	VARCHAR(255)	Название книги
author_id	INTEGER	Внешний ключ на автора
publication_year	INTEGER	Год издания
description	TEXT	Описание книги
total_copies	INTEGER	Общее количество экземпляров
available_copies	INTEGER	Количество доступных экземпляров
cover	VARCHAR(255)	Путь к файлу обложки
Таблица book_categories		
id	INTEGER	Первичный ключ
book_id	INTEGER	Внешний ключ на книгу
category_id	INTEGER	Внешний ключ на категорию
Таблица loans		
id	INTEGER	Первичный ключ
book_id	INTEGER	Внешний ключ на книгу
reader_id	INTEGER	Внешний ключ на читателя
issue_date	DATE	Дата выдачи
due_date	DATE	Плановая дата возврата
return_date	DATE	Фактическая дата возврата
status	VARCHAR(20)	Статус выдачи
Таблица uploaded_files		
id	INTEGER	Первичный ключ
file	VARCHAR(255)	Путь к загруженному файлу
file_type	VARCHAR(50)	Тип файла
uploaded_at	DATETIME	Дата и время загрузки
uploaded_by_id	INTEGER	Внешний ключ на пользователя

Для обеспечения целостности данных необходимо предусмотреть ограничения. Количество доступных экземпляров книги не должно быть отрицательным. Запись о выдаче должна быть связана с существующей книгой и существующим читателем. При оформлении выдачи количество доступных экземпляров книги должно уменьшаться, а при возврате увеличиваться.

2.4 Разработка макетов страниц

На этапе проектирования определены основные страницы веб-приложения. Макеты страниц используются для предварительного описания структуры интерфейса до начала реализации. Они позволяют определить расположение меню, заголовков, форм, таблиц, кнопок и информационных блоков.

Основные страницы приложения:

- страница входа;
- главная страница;
- каталог книг;
- карточка книги;
- страница управления книгами;
- страница управления читателями;
- журнал выдач;
- страница статистики;
- административный раздел.

На рисунке 2.3 представлен макет страницы каталога книг.

Библиотека

Каталог | Книги | Читатели | Журнал выдач | Статистика | Выход

Каталог книг

Поиск и фильтрация

Поиск по названию

Автор

Категория

Доступность

Найти

Название ↑	Автор ↑	Категория ↑	Издания	Доступно ↑	Действия
• ПСС В. И. Ленина	В. И. Ленин	Политэкономика, марксизм-ленинизм, ...	2	5 из 7	Свернуть
5-е изд., Политиздат, 1958–1965		Томы 1–55		3 из 4	Подробнее
4-е изд., Госполитиздат, 1941–1967		Томы 1–45		2 из 3	Подробнее
• Капитал	К. Маркс	Политэкономика, марксизм, ...	3	4 из 6	Развернуть
• Манифест Коммунистической партии	К. Маркс, Ф. Энгельс	Политическая теория, история, ...	2	2 из 3	Развернуть

Страница 1 из 5

В начало | < Пред. | 1 | 2 | 3 | След. > | В конец

Рисунок 2.3 – Макет страницы каталога книг

В макете каталога предусмотрены поиск, фильтрация и сортировка записей. Сортировка выполняется через заголовки столбцов: пользователь может изменить порядок вывода по названию, автору, категории или количеству доступных экземпляров.

Каталог также учитывает наличие разных изданий одного произведения. Обобщенная строка показывает название произведения и суммарную доступность экземпляров, а при раскрытии строки отображаются отдельные издания с указанием издательства, годов выпуска, состава томов и доступного количества экземпляров.

В списке каталога для книги показываются только основные категории. Если книга относится к нескольким категориям, в таблице выводятся первые в списке категории и многоточие, а полный список категорий может отображаться в карточке книги.

На рисунке 2.4 представлен макет страницы журнала выдач.

Библиотека
Каталог | Книги | Читатели | Журнал выдач | Статистика | Выход

Журнал выдач

Панель действий

Оформить выдачу

Статус

Поиск по книге или читателю

Найти

Выгрузить

Книга ↑	Издание ↑	Читатель ↑	Дата выдачи ↑	Срок возврата ↑	Статус ↑	Действия
ПСС В. И. Ленина	5-е изд., т. 1–5	Иванов И. И.	10.05.2026	24.05.2026	Выдана	Просмотр Возврат
Капитал	Т. 1, Политиздат	Петров П. П.	03.05.2026	17.05.2026	Просрочена	Просмотр Возврат
Манифест Коммунистической партии	Изд. 1982 г.	Сидоров С. С.	15.05.2026	29.05.2026	Выдана	Просмотр Возврат

Страница 1 из 4

В начало | < Пред. | 1 | 2 | 3 | След. > | В конец

Рисунок 2.4 – Макет страницы журнала выдач

Макет страницы журнала выдач отражает основной рабочий сценарий библиотекаря: поиск записей о выдаче, фильтрацию по статусу, оформление новой выдачи, просмотр сведений о записи и оформление возврата книги. Для удобства работы в таблице предусмотрена сортировка по основным столбцам: книге, изданию, читателю, дате выдачи, сроку возврата и статусу.

В журнале выдач отображается не только название книги, но и конкретное издание, поскольку выдача оформляется на определенный экземпляр или набор экземпляров. Это позволяет отличать разные издания одного произведения, например разные издания ПСС В. И. Ленина или отдельные тома произведения. В нижней части страницы предусмотрена постраничная навигация с переходом в начало и конец списка.

В результате проектирования определены основные пользователи приложения, описаны функциональные требования, выделены ключевые сущности модели данных и определены страницы пользовательского интерфейса. Полученные проектные решения будут использованы при реализации веб-приложения.

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

Ту би континьюд.

ЗАКЛЮЧЕНИЕ

Ту би континьюд.

СПИСОК ЛИТЕРАТУРЫ

- [1] Koha Library Software [Электронный ресурс]. — [Б. м.] : Koha Community. — URL: <https://koha-community.org/> (дата обращения: 06.05.2026).
- [2] Продукты семейства САБ ИРБИС64 [Электронный ресурс]. — [Б. м.] : Ассоциация ЭБНИТ. — URL: <https://www.elnit.org/produkty-semejstva-sab-irbis64.html> (дата обращения: 06.05.2026).
- [3] 1С:Библиотека. О решении. Возможности [Электронный ресурс]. — [Б. м.] : Фирма «1С». — URL: <https://solutions.1c.ru/catalog/library/features> (дата обращения: 06.05.2026).
- [4] Django. Веб-фреймворк для разработки приложений на языке Python [Электронный ресурс]. — [Б. м.] : Django Software Foundation. — URL: <https://www.djangoproject.com/> (дата обращения: 06.05.2026).
- [5] Документация Flask [Электронный ресурс]. — [Б. м.] : Pallets. — URL: <https://flask.palletsprojects.com/> (дата обращения: 06.05.2026).
- [6] Документация Django. Аутентификация пользователей [Электронный ресурс]. — [Б. м.] : Django Software Foundation. — URL: <https://docs.djangoproject.com/en/6.0/topics/auth/> (дата обращения: 06.05.2026).
- [7] SQLite. Официальный сайт [Электронный ресурс]. — [Б. м.] : SQLite Consortium. — URL: <https://sqlite.org/about.html> (дата обращения: 06.05.2026).
- [8] PostgreSQL. Официальный сайт [Электронный ресурс]. — [Б. м.] : PostgreSQL Global Development Group. — URL: <https://www.postgresql.org/> (дата обращения: 06.05.2026).
- [9] Документация Django. Загрузка файлов [Электронный ресурс]. — [Б. м.] : Django Software Foundation. — URL: <https://docs.djangoproject.com/en/6.0/topics/http/file-uploads/> (дата обращения: 06.05.2026).
- [10] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления [Текст]. — 2017.